

Data Structures and Algorithms Overview

Organizing data efficiently and solving problems step by step

In this guide

- Introduction
- 1. Analyzing Efficiency with Big-O Notation
- 2. Arrays, Linked Lists, Stacks and Queues
- 3. Hash Tables, Trees and Graphs
- 4. Searching and Sorting Algorithms
- 5. Recursion and Problem-Solving Strategies
- 6. Choosing the Right Data Structure
- Conclusion

Introduction

A data structure is a way of organizing data in memory so that it can be used efficiently. An algorithm is a finite, step-by-step procedure for solving a problem. Together they form the foundation of computer science: the right structure paired with the right algorithm can turn a program that takes hours into one that finishes in milliseconds.

This guide surveys the most important ideas: how to measure efficiency with Big-O notation, the classic linear and non-linear data structures, sorting and searching, recursion, and a few problem-solving strategies. Examples are written in Python, but the concepts apply to every language.

Why it matters

- **Performance:** efficient code saves time, money and energy at scale.
- **Problem solving:** common patterns reappear in real systems and technical interviews.
- **Communication:** shared vocabulary such as 'stack', 'hash map' and ' $O(n \log n)$ ' lets engineers discuss designs precisely.

What you will learn

- 1 How to analyze time and space complexity.
- 2 Arrays, linked lists, stacks and queues.
- 3 Hash tables, trees and graphs.
- 4 Searching and sorting algorithms.
- 5 Recursion, divide and conquer, and dynamic programming.
- 6 How to choose a structure for a given problem.

Approach: Do not memorize code. Focus on why each structure is fast at some operations and slow at others. That understanding transfers to any language.

1. Analyzing Efficiency with Big-O Notation

Big-O notation describes how the running time (or memory use) of an algorithm grows as the input size n grows. It ignores constant factors and small terms, focusing on the dominant behavior for large inputs. Two programs may differ by a few milliseconds on ten items, but Big-O reveals which one survives a million.

Notation	Name	Example
$O(1)$	Constant	Reading <code>list[5]</code> , dictionary lookup
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Scanning a list once
$O(n \log n)$	Linearithmic	Merge sort, quicksort (average)
$O(n^2)$	Quadratic	Nested loops, bubble sort
$O(2^n)$	Exponential	Naive recursive Fibonacci

Reading complexity from code

```
def first_item(items):          #  $O(1)$ : one step regardless of size
    return items[0]

def total(items):              #  $O(n)$ : one pass
    s = 0
    for x in items:
        s += x
    return s

def has_duplicate(items):      #  $O(n^2)$ : nested loops
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False
```

Best, average and worst case

Many algorithms behave differently depending on the input. Linear search finds its target immediately in the best case ($O(1)$) but must check every element in the worst case ($O(n)$). Unless stated otherwise, Big-O usually refers to the worst case.

Space complexity

Memory matters too. An algorithm that creates a copy of its input uses $O(n)$ extra space, while one that only keeps a few counters uses $O(1)$. Trading memory for speed, such as caching results in a dictionary, is a very common design decision.

Key points

- Drop constants and lower-order terms: $O(3n + 10)$ is simply $O(n)$.
- Nested loops over the same data usually mean $O(n^2)$.
- Halving the search space each step signals $O(\log n)$.

2. Arrays, Linked Lists, Stacks and Queues

Linear data structures store items in a sequence. They differ in how quickly you can reach, insert and remove elements.

Arrays (Python lists)

An array keeps items in contiguous memory, so reading any position by index is $O(1)$. Inserting or deleting in the middle requires shifting later elements, which costs $O(n)$. Appending at the end is $O(1)$ on average.

Linked lists

A linked list is a chain of nodes, each holding a value and a reference to the next node. Inserting or removing a node is $O(1)$ once you hold a reference to its position, but reaching the k -th element requires walking the chain in $O(k)$. Linked lists shine when you frequently add or remove items at the front.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

# build 1 -> 2 -> 3
head = Node(1, Node(2, Node(3)))
node = head
while node:
    print(node.value)
    node = node.next
```

Stacks (last in, first out)

A stack allows adding and removing only at one end, like a pile of plates. Uses include undo features, browser history, expression evaluation and function call tracking. In Python a list works well: `append()` to push and `pop()` to pop.

Queues (first in, first out)

A queue adds at the back and removes from the front, like a line at a shop. It models task scheduling, print jobs and breadth-first search. Use `collections.deque` for efficient $O(1)$ operations at both ends.

```

from collections import deque

stack = []
stack.append("a"); stack.append("b")
print(stack.pop())      # b    (last in, first out)

queue = deque()
queue.append("a"); queue.append("b")
print(queue.popleft())  # a    (first in, first out)

```

Structure	Access by index	Insert/remove	Typical use
Array/list	$O(1)$	$O(n)$ middle, $O(1)$ end	General storage
Linked list	$O(n)$	$O(1)$ at known position	Frequent insertions
Stack	n/a	$O(1)$ at top	Undo, parsing
Queue (deque)	n/a	$O(1)$ at both ends	Scheduling, BFS

3. Hash Tables, Trees and Graphs

Non-linear structures organize data by relationships rather than by position.

Hash tables

A hash table (Python `dict` and `set`) uses a hash function to convert a key into an array position, giving average $O(1)$ insertion, lookup and deletion. When two keys land in the same slot, a collision occurs, and the table resolves it with techniques such as chaining or probing. Hash tables are the go-to tool for counting, caching and detecting duplicates.

```

def two_sum(nums, target):
    seen = {}                # value -> index
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i

print(two_sum([2, 7, 11, 15], 9))  # (0, 1)    in  $O(n)$  time

```

Trees

A tree is a hierarchy of nodes with one root and no cycles. File systems, HTML documents and organization charts are all trees. A **binary search tree (BST)** keeps smaller values in the left subtree and larger values in the right, allowing search in $O(\log n)$ when the tree is balanced.

```

class TreeNode:
    def __init__(self, key):
        self.key, self.left, self.right = key, None, None

def insert(root, key):
    if root is None:
        return TreeNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder(root):
    # yields keys in sorted order
    if root:
        yield from inorder(root.left)
        yield root.key
        yield from inorder(root.right)

```

- **Traversals:** in-order, pre-order, post-order (depth-first) and level-order (breadth-first).
- **Heaps:** trees that always keep the smallest (or largest) item on top; used for priority queues.

Graphs

A graph consists of vertices (nodes) connected by edges, which may be directed or weighted. Social networks, road maps and the web are graphs. The usual representation is an adjacency list, a dictionary mapping each node to its neighbors.

```

graph = {"A": ["B", "C"], "B": ["D"], "C": ["D"], "D": []}

from collections import deque
def bfs(start):
    seen, order, q = {start}, [], deque([start])
    while q:
        node = q.popleft()
        order.append(node)
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb); q.append(nb)
    return order

print(bfs("A"))    # ['A', 'B', 'C', 'D']

```

Key points

- Use a hash table for fast lookups by key.
- Use a tree for hierarchical data or ordered searching.
- Use a graph whenever the data is about connections.

4. Searching and Sorting Algorithms

Searching and sorting are the most studied algorithmic problems because they underlie so many others. Sorted data enables faster searching, easier duplicate detection and clearer reporting.

Linear and binary search

Linear search checks each item in turn ($O(n)$) and works on any list. Binary search requires sorted data but repeatedly halves the range, giving $O(\log n)$: searching one billion sorted items takes only about 30 comparisons.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9, 11], 7))    # 3
```

Common sorting algorithms

Algorithm	Average	Worst	Idea
Bubble sort	$O(n^2)$	$O(n^2)$	Swap adjacent items that are out of order
Insertion sort	$O(n^2)$	$O(n^2)$	Insert each item into the sorted part; fast on nearly sorted data
Merge sort	$O(n \log n)$	$O(n \log n)$	Split, sort halves, merge; stable
Quicksort	$O(n \log n)$	$O(n^2)$	Partition around a pivot; fast in practice
Heap sort	$O(n \log n)$	$O(n \log n)$	Build a heap, extract repeatedly

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

print(merge_sort([5, 2, 9, 1, 7])) # [1, 2, 5, 7, 9]
```

In practice: Use the built-in `sorted()` or `list.sort()`. Python's Timsort is highly optimized, stable and runs in $O(n \log n)$.

Key points

- Binary search needs sorted input.
- Comparison sorts cannot beat $O(n \log n)$ in general.
- A stable sort keeps equal items in their original order.

5. Recursion and Problem-Solving Strategies

Recursion means a function solves a problem by calling itself on a smaller version of the same problem. Every recursive function needs a **base case** that stops the calls and a **recursive case** that moves toward it.

```
def factorial(n):
    if n <= 1:                # base case
        return 1
    return n * factorial(n - 1) # recursive case

print(factorial(5))          # 120
```

Each call is stored on the call stack, so very deep recursion can exhaust memory (Python raises `RecursionError` at about 1000 levels). Some recursive solutions can be rewritten as loops.

Divide and conquer

Split a problem into independent subproblems, solve each, and combine the results. Merge sort and binary search follow this pattern.

Dynamic programming

When subproblems overlap, computing them again and again wastes time. Dynamic programming stores each result so it is computed only once, a technique called memoization. The naive Fibonacci function takes $O(2^n)$ time, but caching makes it $O(n)$.

```

from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50))    # 12586269025, instantly

```

Greedy algorithms and other patterns

- **Greedy:** always take the locally best choice (for example, making change with the largest coin first). Fast but only correct for some problems.
- **Two pointers:** move two indexes through a sorted list to find pairs.
- **Sliding window:** maintain a moving range to solve subarray problems in $O(n)$.
- **Backtracking:** build solutions step by step and undo choices that fail (Sudoku, permutations).

A method for attacking new problems

- 1 Restate the problem and work a small example by hand.
- 2 Start with a simple brute-force solution and note its complexity.
- 3 Look for repeated work, sorted order or a useful structure (hash map, heap).
- 4 Improve the solution, then test edge cases: empty input, one element, duplicates.

6. Choosing the Right Data Structure

Most design decisions reduce to one question: which operations will happen most often? The table below summarizes typical costs; use it as a quick decision guide.

Structure	Lookup	Insert	Delete	Ordered?
Array/list	$O(n)$ search, $O(1)$ index	$O(1)$ end, $O(n)$ middle	$O(n)$	By position
Hash table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes (sorted)
Heap	$O(1)$ min/max	$O(\log n)$	$O(\log n)$	Partially
Stack/Queue	n/a	$O(1)$	$O(1)$	LIFO / FIFO

Scenario guide

- **Count how often each word appears:** hash table (`dict` or `Counter`).
- **Process tasks in arrival order:** queue.
- **Undo the last action:** stack.
- **Always fetch the highest-priority job:** heap / priority queue.
- **Find shortest route between locations:** graph plus BFS or Dijkstra's algorithm.

- **Keep items sorted with fast insertion:** balanced tree (or a sorted container library).
- **Check if an item was seen before:** set.

Practical advice

Clarity usually beats cleverness. Choose the simplest structure that meets performance needs, measure before optimizing, and remember that for small inputs any approach works. Complexity analysis matters most when data is large or when an operation runs inside a loop.

Common mistakes

- Using a list for membership tests on large data ($O(n)$) instead of a set ($O(1)$).
- Repeatedly inserting at the front of a list, which shifts every element.
- Ignoring worst cases such as sorted input for a naive quicksort.
- Forgetting the base case in recursion.

Conclusion

Data structures and algorithms give you the tools to reason about efficiency and to design programs that scale. You have seen how Big-O notation describes growth, how linear and non-linear structures trade off different operations, how the classic search and sort algorithms work, and how recursion and dynamic programming break large problems into manageable pieces.

Key takeaways

- Analyze algorithms by how they scale, not by how fast they run on tiny inputs.
- Hash tables, arrays, trees, heaps and graphs each excel at particular operations.
- Sorting enables binary search and simplifies many problems.
- Recursion needs a base case; memoization removes repeated work.
- Choose the structure by asking which operations dominate.

Suggested next steps

- 1 Implement each structure from scratch once to internalize how it works.
- 2 Solve 2-3 practice problems per week on LeetCode, HackerRank or Codeforces.
- 3 Study advanced topics: balanced trees, tries, union-find, Dijkstra and topological sort.
- 4 Read a textbook such as Introduction to Algorithms (CLRS) or Grokking Algorithms.

Remember: Consistent practice beats cramming. Understand the pattern behind a solution so you can recognize it in new problems.